

Watch It Run: Monitoring, Logging & Analytics

2026-04-19 · brisk pomegranate Haubentaucher

YOU CANNOT IMPROVE WHAT YOU CANNOT MEASURE.

From Debugging to Observability

YOU ALREADY KNOW `journalctl` for reading logs and debugging crashes (Block 5). Now the question changes: your service is running and nobody is watching the terminal. How do you know it is working well? Three pillars:

Logs Structured, queryable records of what happened.

Monitoring Automated checks that alert you when something breaks.

Analytics Patterns in your data that reveal how the system behaves over time.

Structured Logging

PLAIN STRINGS ARE NOT ENOUGH. `print("got request")` tells you something happened. It does not tell you what, when, from whom, how long it took, or what the result was. You cannot filter, aggregate, or search plain strings at scale.

STRUCTURED LOGGING means every log line is a JSON dict:

```
{
  "timestamp": "2026-04-05T14:23:01Z",
  "level": "INFO",
  "endpoint": "/classify",
  "request_id": "abc-123",
  "latency_ms": 42,
  "predicted_class": "7",
  "confidence": 0.94,
  "model_version": "v1"
}
```

IN PYTHON, configure structured logging:

```
import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    def format(self, record):
        log_data = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "message": record.getMessage(),
        }
        # Add extra fields if present
        for key in ["endpoint", "request_id",
                   "latency_ms", "predicted_class",
                   "confidence", "model_version"]:
            if hasattr(record, key):
                log_data[key] = getattr(record, key)
        return json.dumps(log_data)
```

Structured logs are parseable by machines, readable by humans, and queryable with `jq` (<https://jqlang.github.io/jq/>) or piped into any log aggregation tool. Python logging: <https://docs.python.org/3/library/logging.html>

Exercise: Add Structured JSON Logging

CONFIGURE THE PYTHON logging MODULE with a JSON formatter using the class shown in the theory above. Every request should log: timestamp, level, endpoint, request_id, latency_ms, predicted_class, confidence, model_version.

If your VM is broken, restore the B8-complete snapshot. At the end of this session, take a snapshot named B9-complete.

The /health Endpoint Upgraded

BLOCK 5's /health confirmed the process is running. Now distinguish two levels:

Liveness Is the process running? Returns 200 if yes.

Readiness Can it serve requests? Is the DB reachable? Is the model loaded?

```
@app.get("/health")
def health():
    db_ok = check_db_connection()
    model_ok = _pipeline is not None
    status = "ok" if (db_ok and model_ok) else "degraded"
    return {"status": status, "model_version": MODEL_VERSION}
```

What to expose externally vs what to keep internal: the public /health endpoint returns "ok" or "degraded". It does not expose DB passwords, stack traces, or version details. Detailed diagnostics are logged internally.

Exercise: Upgrade /health

CHECK DB CONNECTIVITY AND MODEL LOAD STATUS. Return
{"status": "ok"} externally; log detailed diagnostics internally.

Monitoring with Discord Webhooks

NO NEW TOOLS TO INSTALL. A cron job curls /health every 5 minutes. If the response is not "ok", it sends a Discord webhook notification. Your service talks to you when something breaks, on your phone, in your pocket.

```
#!/bin/bash
# scripts/health_check.sh
HEALTH=$(curl -s http://localhost:8000/health)
STATUS=$(echo $HEALTH | jq -r '.status')

if [ "$STATUS" != "ok" ]; then
    curl -H "Content-Type: application/json" \
        -d "{\"content\": \"PixelWise ALERT: $HEALTH\"}" \
        $DISCORD_WEBHOOK_URL
fi
```

REGISTER AS A CRON JOB:

```
* /5 * * * * /opt/pixelwise/scripts/health_check.sh
```

The Discord webhook URL is stored in .env. It is a secret that grants posting access to your channel. The cron job runs the script; the script talks to Discord when there is a problem. Discord Webhooks: <https://discord.com/developers/docs/resources/webhook>

Exercise: Set Up Discord Webhook Monitoring

WIRE THE ALERTING PATH END TO END. Work through the steps below:

- Create a Discord server and webhook (Settings → Integrations → Webhooks).
- Add `DISCORD_WEBHOOK_URL` to `.env` and `.env.example`.
- Write `scripts/health_check.sh` using the script from the theory section.
- Register the cron job:

```
crontab -e
# Add:
*/5 * * * * /opt/pixelwise/scripts/health_check.sh
```

- Test it: stop the `systemd` service and wait. Does Discord ping?

```
sudo systemctl stop pixelwise
# Wait up to 5 minutes, check Discord
sudo systemctl start pixelwise
```

Event-Driven Notifications

THE DISCORD PATTERN GENERALISES. Any event worth knowing about can trigger a notification. A second example: notify when the model produces a low-confidence prediction (below a threshold).

```
# In the classify endpoint
if result["confidence"] < CONFIDENCE_THRESHOLD:
    send_discord_alert(
        f"Low confidence: {result['prediction']} "
        f"({result['confidence']:.2f})"
    )
```

IF THESE SPIKE, the model may be struggling with inputs it was not trained on, a real signal, not noise. This is the natural step toward model monitoring: tracking confidence over time, spotting distribution shifts, detecting when a model goes stale.

Exercise: Add a Low-Confidence Alert

IN THE CLASSIFY ENDPOINT, send a Discord notification when confidence falls below a threshold (e.g., 0.5). Test it by drawing an ambiguous digit. Does Discord ping?

Exercise: Add a Canvas-Clear Notification

ADD A SECOND DISCORD NOTIFICATION that fires when a user clears the drawing field. Add an API endpoint (e.g., POST /api/events/clear) that the frontend calls on canvas clear. Discuss: what does a spike in clears tell you about model performance?

Analytics

EVERYTHING THAT HAPPENS IN THE APP IS A POTENTIAL DATA POINT:

Backend signals Prediction counts, confidence values, model version usage, already stored in PostgreSQL from Block 6.

Frontend signals Page loads, drawings submitted, canvas clears, time spent drawing. Any user interaction can be tracked by the frontend and sent to the API.

Analytics Concepts

The /analytics Endpoint

PROVIDED AS BOILERPLATE. It returns prediction counts, average confidence, and model version breakdown. Students hit the endpoint and see the data. The lesson is not how to write the queries, but what the numbers tell you about your app.

```
@app.get("/api/analytics")
def analytics():
    # Boilerplate: queries predictions table
    return {
        "total_predictions": ...,
        "predictions_per_class": {...},
        "avg_confidence": ...,
        "model_versions": {...}
    }
```

These concepts are how product teams understand user behaviour at scale: **funnels** (open page → draw → submit → view result, where do users drop off?), **retention** (do users come back?), **sessions** (grouping events into a single visit, recall the session IDs from Block 7's URL design).

In production, analytics panels are never public. They reveal usage patterns and internal metrics. They belong behind authentication on a separate internal route (e.g., `/admin/analytics`).

Frontend Analytics

INSTRUMENT THE FRONTEND to count user interactions. A small panel in the UI displays: page loads, drawings submitted, canvas clears. These are frontend-only counters, simple JavaScript variables incremented on each event.

CANVAS CLEARS ARE A SIGNAL TOO. When a user clears the drawing field, it often means the prediction was wrong or the user wants to try again. This is the natural step to wire in model analytics. A spike in clears correlated with low-confidence predictions tells you the model is struggling with certain inputs.

Exercise: Deploy the /analytics Endpoint

USE THE PROVIDED BOILERPLATE. Hit it with curl and interpret the results:

```
curl https://192.168.56.11/api/analytics -k \
-H "X-API-Key: your-key"
```

READ THE NUMBERS. Which digit is drawn most? Is one model version more confident than the other?

Exercise: Add Frontend Analytics

INSTRUMENT THE STARTER CODE to count page loads, drawings submitted, and canvas clears. Display these in a small panel in the UI. Discuss: what would you track if this were a real product?

Exercise: Commit

STAGE AND PUSH the new logging, alerting, and analytics work:

```
git add app/ scripts/ frontend/ .env.example
git commit -m "Add structured logging, Discord alerts, analytics"
git push
```

Exercise: Take a Snapshot

SNAPSHOT BOTH VMs and name them B9-complete.

A NOTE ON WHAT COMES NEXT. With the service observable end to end, the focus shifts from watching the system to scaling and hardening it. The next block carries that thread forward.

Security Thread

LOG SANITIZATION. Never log raw pixel data or user identifiers unnecessarily. The /health endpoint confirms the system is alive but does not expose internal state (DB passwords, stack traces, version details) to unauthenticated callers.

Self-Reflection and Recap

REFLECTION QUESTIONS:

- What is the difference between plain-text logging and structured logging? Why does it matter?
- Why do we monitor with Discord webhooks instead of installing a monitoring platform?
- What does a spike in low-confidence predictions tell you about your model?
- Why should analytics endpoints never be public?
- What would a spike in canvas clears correlated with low-confidence predictions indicate?

MILESTONE: PixelWise is fully observable. Logs are structured and queryable. Discord alerts on health check failure and low-confidence predictions. Analytics endpoint and frontend counters show usage patterns. Students can answer: “how is my app being used, and how is my model performing?”